

RNN

December 17, 2025

```
[94]: from tensorflow import keras
import numpy as np
import matplotlib.pyplot as plt
import os
```

```
[95]: def generate_time_series(batch_size, n_step):
    freq1, freq2, offsets1, offsets2 = np.random.rand(4, batch_size, 1)
    time = np.linspace(0, 1, n_step)
    series = 0.5 * np.sin((time - offsets1) * (freq1 * 10 + 10)) # Wave 1
    series += 0.2 * np.sin((time - offsets2) * (freq2 * 20 + 20)) # Wave 2
    series += 0.1 * (np.random.rand(batch_size, n_step) - 0.5) # Noise
    return series[..., np.newaxis].astype(np.float32)
```

```
[96]: nstep = 50
series = generate_time_series(10000, nstep + 1)
X_train, y_train = series[:7000, :nstep], series[:7000, -1]
X_valid, y_valid = series[7000:9000, :nstep], series[7000:9000, -1]
X_test, y_test = series[9000:, :nstep], series[9000:, -1]
```

```
[97]: Ypred = X_valid[:, -1]
np.mean(keras.losses.mse(y_valid, Ypred).numpy())
```

```
[97]: np.float32(0.021153366)
```

```
[98]: epochs = 10
model = keras.models.Sequential([
    keras.layers.Flatten(input_shape=[nstep, 1]),
    keras.layers.Dense(1)
])
model.summary()
```

Model: "sequential_14"

Layer (type)	Output Shape	Param #
flatten_4 (Flatten)	(None, 50)	0

dense_8 (Dense)

(None, 1)

51

Total params: 51 (204.00 B)

Trainable params: 51 (204.00 B)

Non-trainable params: 0 (0.00 B)

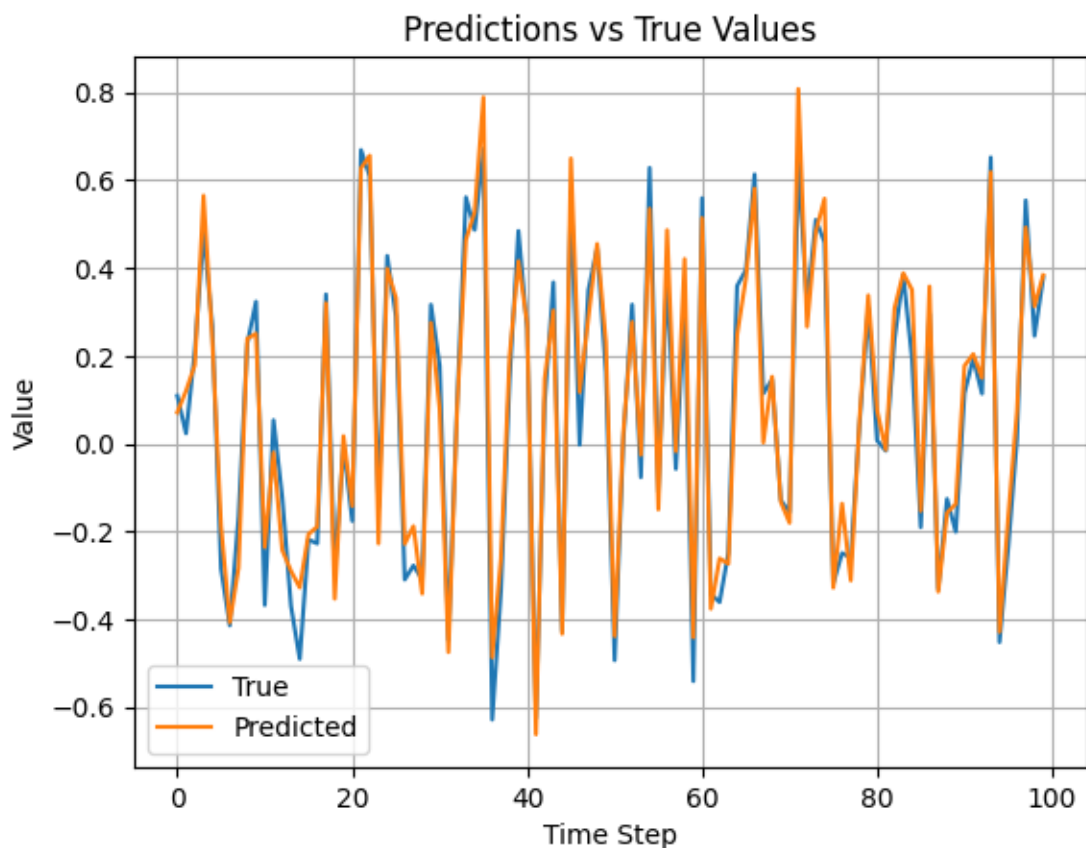
```
[99]: model.compile(loss=keras.losses.mse,
                optimizer=keras.optimizers.Adam(learning_rate=1e-3))
history = model.fit(X_train, y_train, epochs=epochs, validation_data=(X_valid,
↪y_valid))
```

```
Epoch 1/10
219/219          1s 3ms/step -
loss: 0.1193 - val_loss: 0.0295
Epoch 2/10
219/219          1s 3ms/step -
loss: 0.0175 - val_loss: 0.0130
Epoch 3/10
219/219          1s 5ms/step -
loss: 0.0098 - val_loss: 0.0089
Epoch 4/10
219/219          1s 2ms/step -
loss: 0.0075 - val_loss: 0.0073
Epoch 5/10
219/219          1s 3ms/step -
loss: 0.0065 - val_loss: 0.0065
Epoch 6/10
219/219          1s 3ms/step -
loss: 0.0060 - val_loss: 0.0060
Epoch 7/10
219/219          1s 3ms/step -
loss: 0.0056 - val_loss: 0.0056
Epoch 8/10
219/219          1s 3ms/step -
loss: 0.0053 - val_loss: 0.0053
Epoch 9/10
219/219          1s 3ms/step -
loss: 0.0050 - val_loss: 0.0051
Epoch 10/10
219/219          1s 3ms/step -
loss: 0.0048 - val_loss: 0.0049
```

```
[100]: y_pred = model.predict(X_valid)
plt.plot(y_valid[:100], label='True')
plt.plot(y_pred[:100], label='Predicted')
plt.title('Predictions vs True Values')
plt.xlabel('Time Step')
plt.ylabel('Value')
plt.legend()
plt.grid()
plt.show()
np.mean(keras.losses.mse(y_valid, y_pred).numpy())
```

63/63

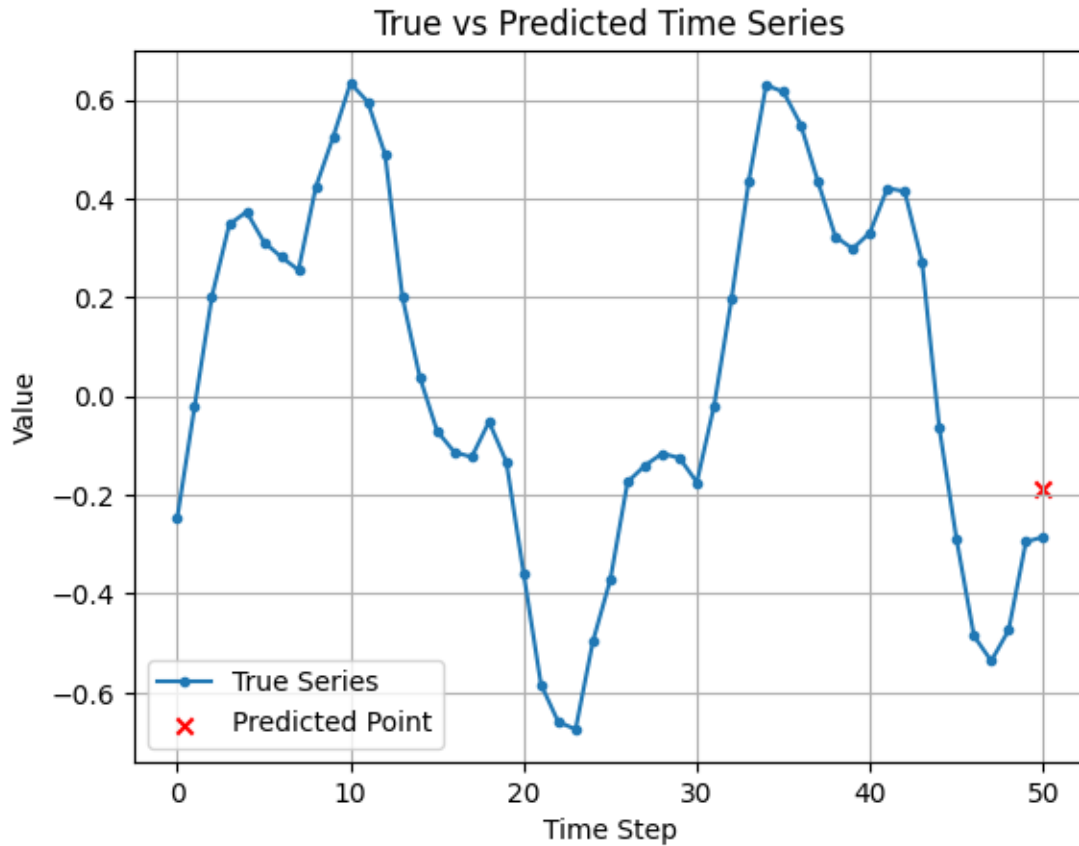
0s 3ms/step



```
[100]: np.float32(0.004871212)
```

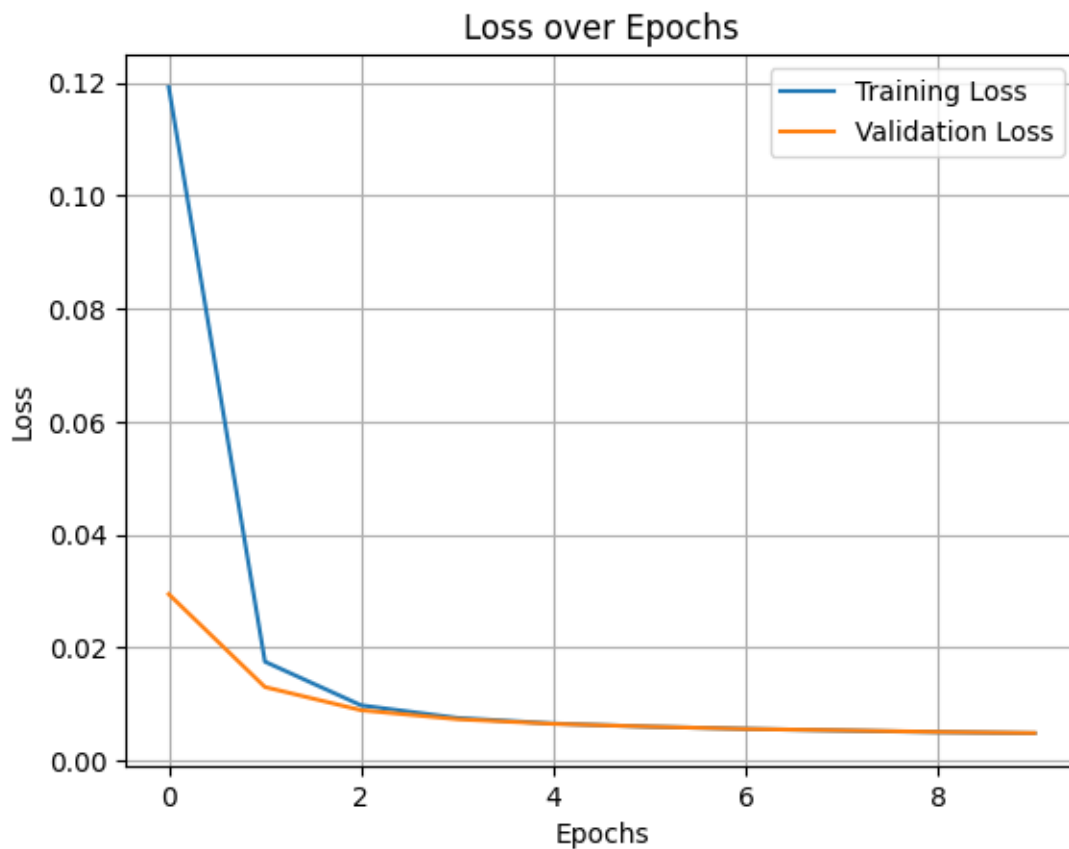
```
[101]: plt.plot(np.hstack((X_valid[5,:,0],y_valid[5])),marker = '.', label='True_
↳Series')
plt.scatter([nstep], y_pred[5], marker = 'x',color='red', label='Predicted_
↳Point')
plt.title('True vs Predicted Time Series')
```

```
plt.xlabel('Time Step')
plt.ylabel('Value')
plt.legend()
plt.grid()
plt.show()
```



```
[102]: def plot_results(history):

    # Plot loss
    plt.plot(history.history['loss'], label='Training Loss')
    plt.plot(history.history['val_loss'], label='Validation Loss')
    plt.title('Loss over Epochs')
    plt.xlabel('Epochs')
    plt.ylabel('Loss')
    plt.legend()
    plt.grid(True)
    plt.show()
    plot_results(history)
```



0.1 RNN

```
[103]: model = keras.models.Sequential([
        keras.layers.SimpleRNN(1, input_shape=[None, 1]),
    ])
model.summary()
```

Model: "sequential_15"

Layer (type)	Output Shape	Param #
simple_rnn_14 (SimpleRNN)	(None, 1)	3

Total params: 3 (12.00 B)

Trainable params: 3 (12.00 B)

Non-trainable params: 0 (0.00 B)

```
[104]: model.compile(loss=keras.losses.mse,  
                optimizer=keras.optimizers.Adam(learning_rate=1e-2))  
model.fit(X_train, y_train, epochs=epochs, validation_data=(X_valid, y_valid))
```

```
Epoch 1/10  
219/219          4s 11ms/step -  
loss: 0.0651 - val_loss: 0.0278  
Epoch 2/10  
219/219          2s 10ms/step -  
loss: 0.0208 - val_loss: 0.0177  
Epoch 3/10  
219/219          3s 12ms/step -  
loss: 0.0150 - val_loss: 0.0140  
Epoch 4/10  
219/219          2s 9ms/step -  
loss: 0.0125 - val_loss: 0.0123  
Epoch 5/10  
219/219          2s 8ms/step -  
loss: 0.0115 - val_loss: 0.0116  
Epoch 6/10  
219/219          2s 11ms/step -  
loss: 0.0111 - val_loss: 0.0115  
Epoch 7/10  
219/219          2s 9ms/step -  
loss: 0.0110 - val_loss: 0.0114  
Epoch 8/10  
219/219          3s 11ms/step -  
loss: 0.0110 - val_loss: 0.0115  
Epoch 9/10  
219/219          3s 12ms/step -  
loss: 0.0110 - val_loss: 0.0115  
Epoch 10/10  
219/219          2s 10ms/step -  
loss: 0.0111 - val_loss: 0.0115
```

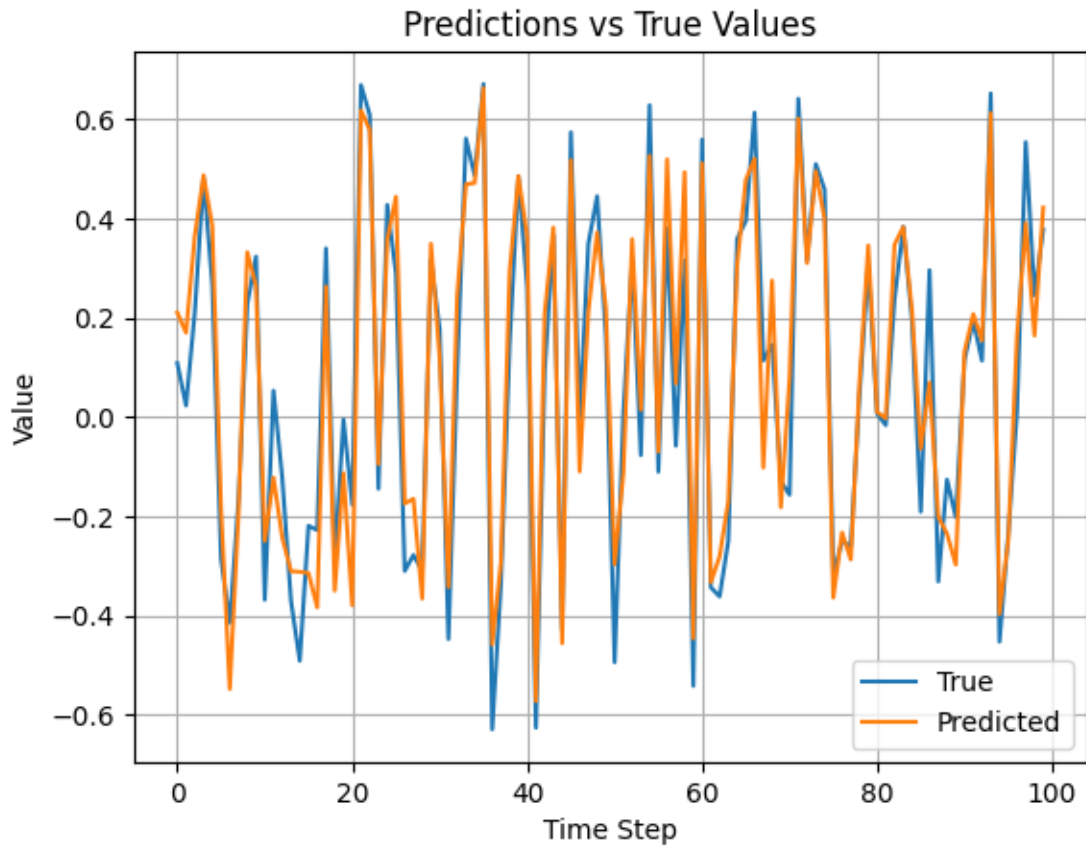
```
[104]: <keras.src.callbacks.history.History at 0x269e7e3f2b0>
```

```
[105]: y_pred = model.predict(X_valid)  
plt.plot(y_valid[:100], label='True')  
plt.plot(y_pred[:100], label='Predicted')  
plt.title('Predictions vs True Values')  
plt.xlabel('Time Step')  
plt.ylabel('Value')  
plt.legend()  
plt.grid()
```

```
plt.show()
np.mean(keras.losses.mse(y_valid, y_pred).numpy())
```

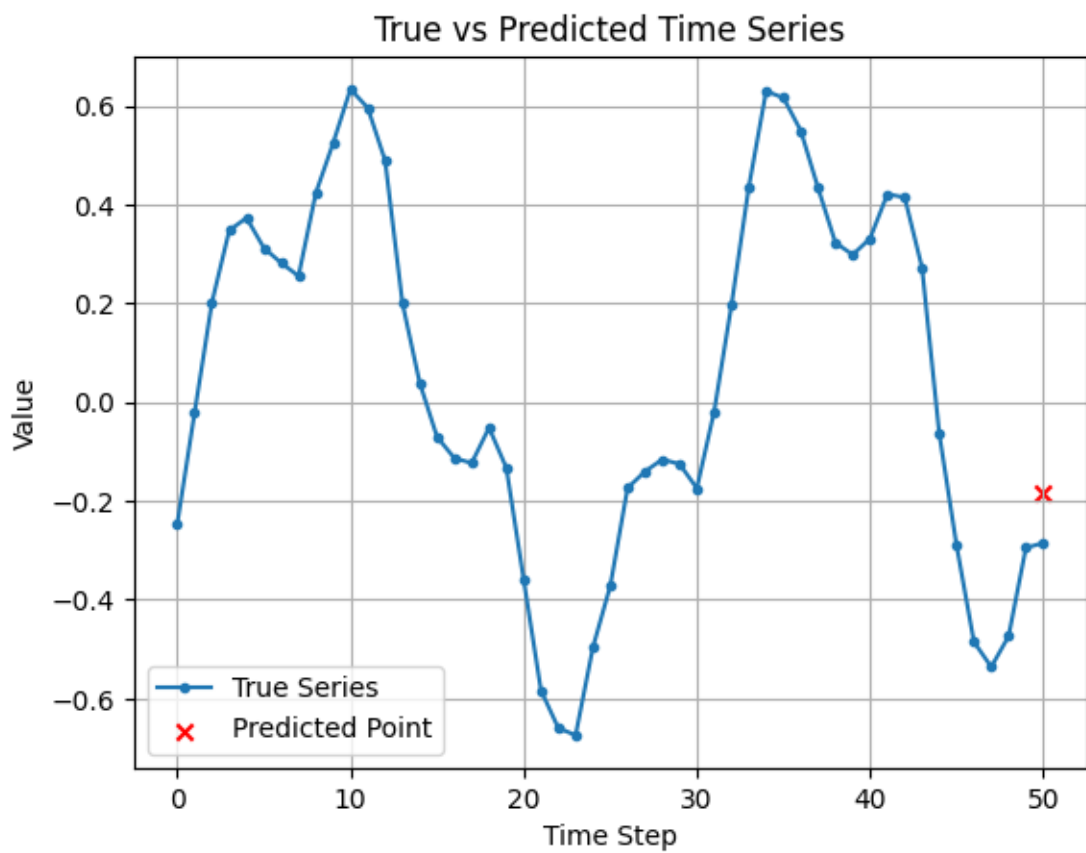
63/63

1s 5ms/step

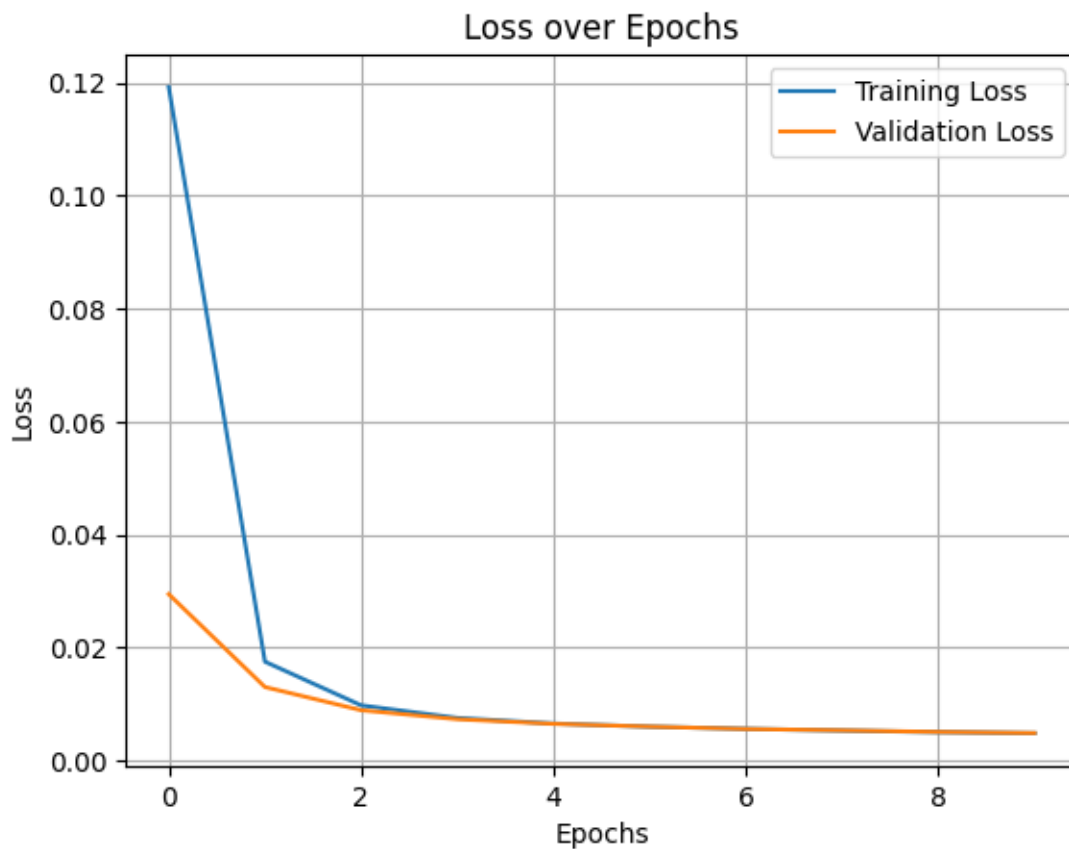


```
[105]: np.float32(0.01148476)
```

```
[106]: plt.plot(np.hstack((X_valid[5,:,0],y_valid[5])),marker = '.', label='True_
↳Series')
plt.scatter([nstep], y_pred[5], marker = 'x',color='red', label='Predicted_
↳Point')
plt.title('True vs Predicted Time Series')
plt.xlabel('Time Step')
plt.ylabel('Value')
plt.legend()
plt.grid()
plt.show()
```



```
[107]: plot_results(history)
```



```
[108]: model = keras.models.Sequential([
    keras.layers.SimpleRNN(20, return_sequences=True, input_shape=[None, 1]),
    keras.layers.SimpleRNN(20),
    keras.layers.Dense(1)
])
model.summary()
```

Model: "sequential_16"

Layer (type)	Output Shape	Param #
simple_rnn_15 (SimpleRNN)	(None, None, 20)	440
simple_rnn_16 (SimpleRNN)	(None, 20)	820
dense_9 (Dense)	(None, 1)	21

Total params: 1,281 (5.00 KB)

Trainable params: 1,281 (5.00 KB)

Non-trainable params: 0 (0.00 B)

```
[109]: model.compile(loss=keras.losses.mse,  
                  optimizer=keras.optimizers.Adam(learning_rate=1e-2))  
model.fit(X_train, y_train, epochs=epochs, validation_data=(X_valid, y_valid))
```

```
Epoch 1/10  
219/219      8s 18ms/step -  
loss: 0.0097 - val_loss: 0.0037  
Epoch 2/10  
219/219      3s 15ms/step -  
loss: 0.0039 - val_loss: 0.0037  
Epoch 3/10  
219/219      4s 16ms/step -  
loss: 0.0038 - val_loss: 0.0036  
Epoch 4/10  
219/219      5s 15ms/step -  
loss: 0.0037 - val_loss: 0.0037  
Epoch 5/10  
219/219      3s 15ms/step -  
loss: 0.0037 - val_loss: 0.0035  
Epoch 6/10  
219/219      5s 15ms/step -  
loss: 0.0037 - val_loss: 0.0041  
Epoch 7/10  
219/219      5s 17ms/step -  
loss: 0.0038 - val_loss: 0.0043  
Epoch 8/10  
219/219      5s 16ms/step -  
loss: 0.0036 - val_loss: 0.0038  
Epoch 9/10  
219/219      5s 15ms/step -  
loss: 0.0036 - val_loss: 0.0034  
Epoch 10/10  
219/219      5s 16ms/step -  
loss: 0.0037 - val_loss: 0.0035
```

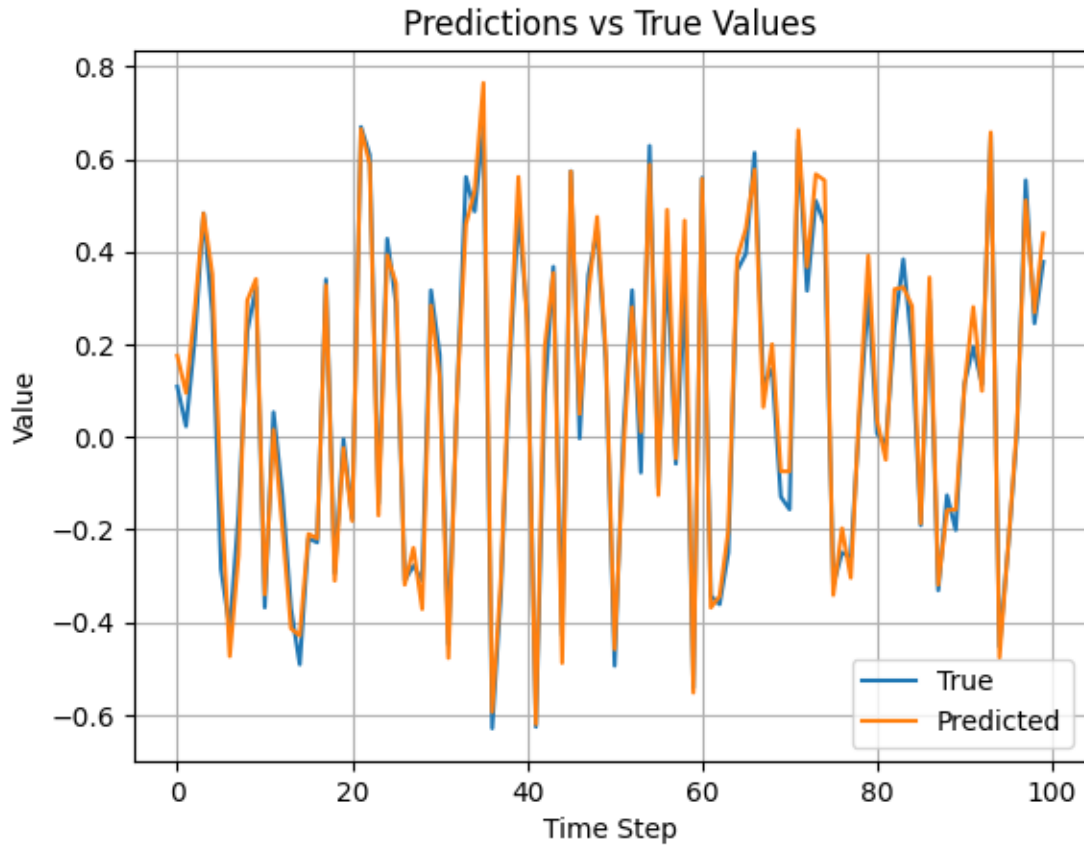
```
[109]: <keras.src.callbacks.history.History at 0x269f37ce740>
```

```
[110]: y_pred = model.predict(X_valid)  
plt.plot(y_valid[:100], label='True')  
plt.plot(y_pred[:100], label='Predicted')
```

```
plt.title('Predictions vs True Values')
plt.xlabel('Time Step')
plt.ylabel('Value')
plt.legend()
plt.grid()
plt.show()
np.mean(keras.losses.mse(y_valid, y_pred).numpy())
```

63/63

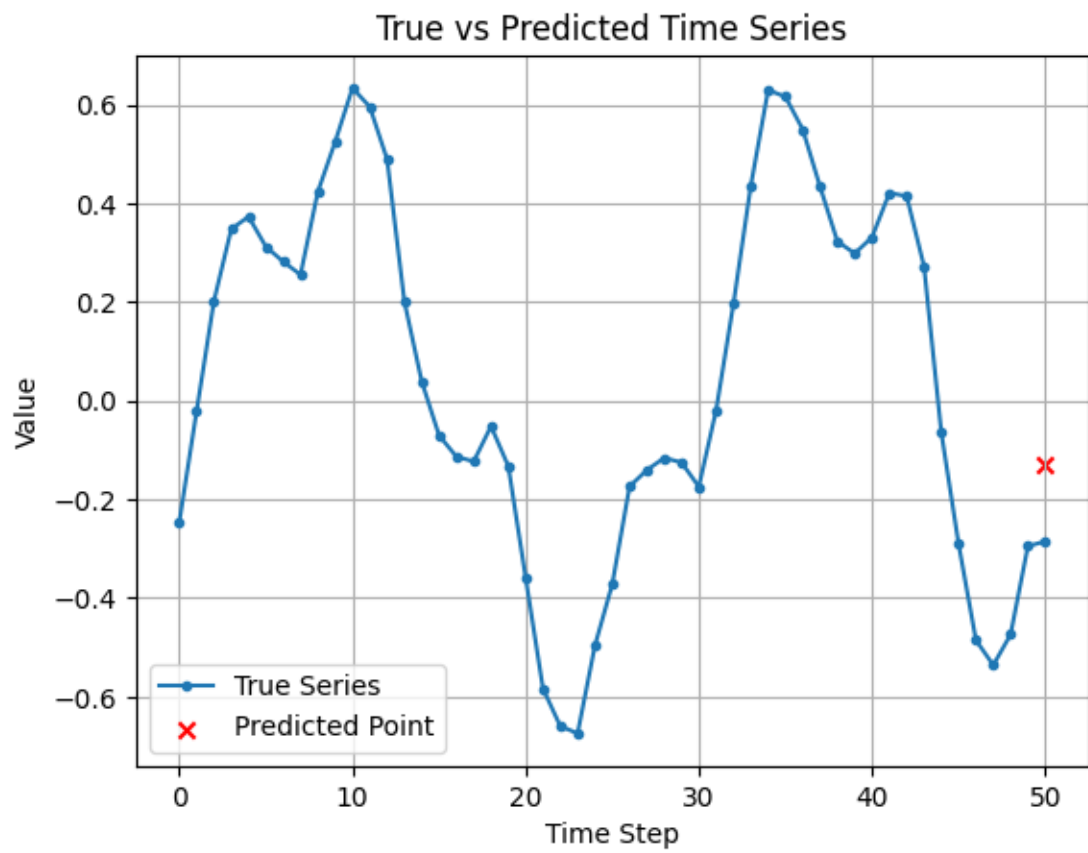
1s 14ms/step



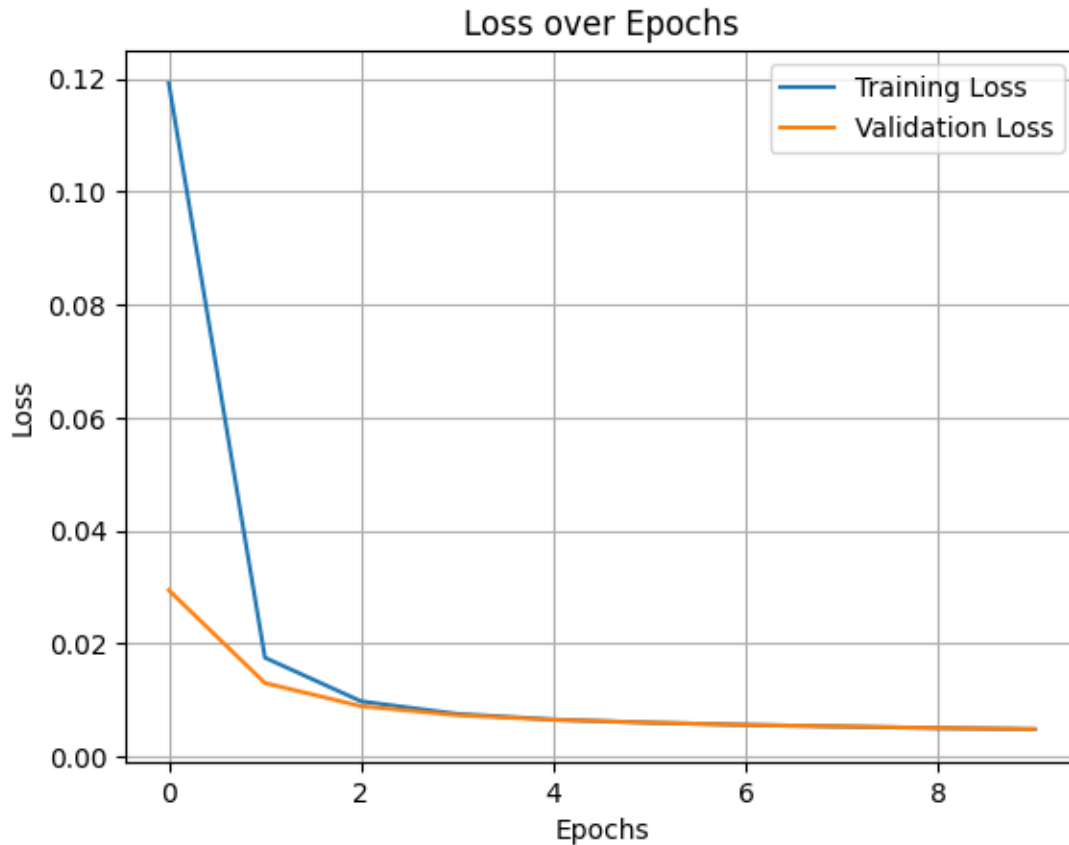
[110]: np.float32(0.0035452142)

```
[ ]: plt.plot(np.hstack((X_valid[5,:,0],y_valid[5])),marker = '.', label='True_
↳Series')
plt.scatter([nstep], y_pred[5], marker = 'x',color='red', label='Predicted_
↳Point')
plt.title('True vs Predicted Time Series')
plt.xlabel('Time Step')
plt.ylabel('Value')
plt.legend()
```

```
plt.grid()  
plt.show()
```



```
[112]: plot_results(history)
```



```
[113]: series = generate_time_series(10000, nstep + 10)
X_train, y_train = series[:7000, :nstep], series[:7000, -10:, 0]
X_valid, y_valid = series[7000:9000, :nstep], series[7000:9000, -10:, 0]
X_test, y_test = series[9000:, :nstep], series[9000:, -10:, 0]

[114]: model = keras.models.Sequential([
    keras.layers.SimpleRNN(20, return_sequences=True, input_shape=[None, 1]),
    keras.layers.SimpleRNN(20),
    keras.layers.Dense(10)
])
model.summary()
```

Model: "sequential_17"

Layer (type)	Output Shape	Param #
simple_rnn_17 (SimpleRNN)	(None, None, 20)	440
simple_rnn_18 (SimpleRNN)	(None, 20)	820

dense_10 (Dense)

(None, 10)

210

Total params: 1,470 (5.74 KB)

Trainable params: 1,470 (5.74 KB)

Non-trainable params: 0 (0.00 B)

```
[115]: model.compile(loss=keras.losses.mse,  
                  optimizer=keras.optimizers.Adam(learning_rate=1e-2))  
model.fit(X_train, y_train, epochs=epochs, validation_data=(X_valid, y_valid))
```

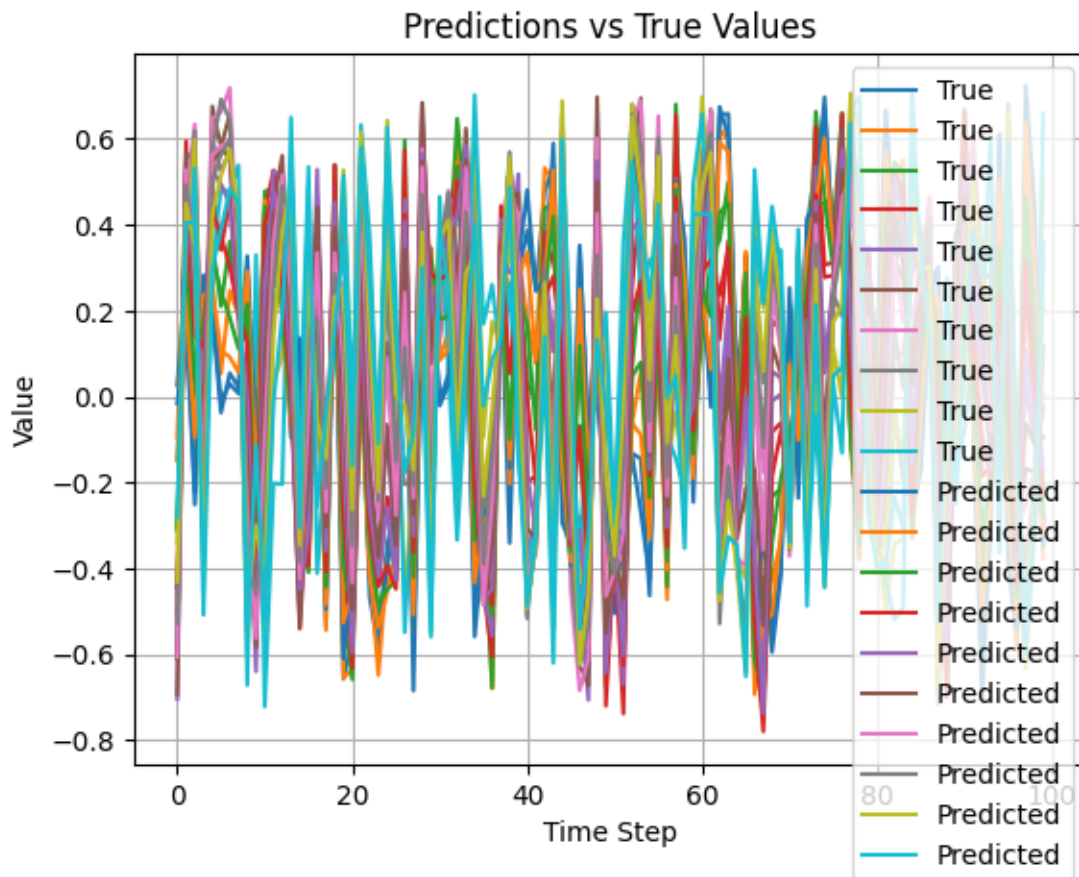
```
Epoch 1/10  
219/219          7s 19ms/step -  
loss: 0.0432 - val_loss: 0.0263  
Epoch 2/10  
219/219          3s 15ms/step -  
loss: 0.0248 - val_loss: 0.0219  
Epoch 3/10  
219/219          3s 16ms/step -  
loss: 0.0169 - val_loss: 0.0138  
Epoch 4/10  
219/219          3s 15ms/step -  
loss: 0.0168 - val_loss: 0.0126  
Epoch 5/10  
219/219          4s 17ms/step -  
loss: 0.0146 - val_loss: 0.0123  
Epoch 6/10  
219/219          4s 17ms/step -  
loss: 0.0146 - val_loss: 0.0115  
Epoch 7/10  
219/219          3s 16ms/step -  
loss: 0.0147 - val_loss: 0.0138  
Epoch 8/10  
219/219          4s 16ms/step -  
loss: 0.0138 - val_loss: 0.0147  
Epoch 9/10  
219/219          4s 16ms/step -  
loss: 0.0138 - val_loss: 0.0175  
Epoch 10/10  
219/219          3s 16ms/step -  
loss: 0.0144 - val_loss: 0.0145
```

[115]: <keras.src.callbacks.history.History at 0x269f60784c0>

```
[116]: y_pred = model.predict(X_valid)
plt.plot(y_valid[:100], label='True')
plt.plot(y_pred[:100], label='Predicted')
plt.title('Predictions vs True Values')
plt.xlabel('Time Step')
plt.ylabel('Value')
plt.legend()
plt.grid()
plt.show()
np.mean(keras.losses.mse(y_valid, y_pred).numpy())
```

63/63

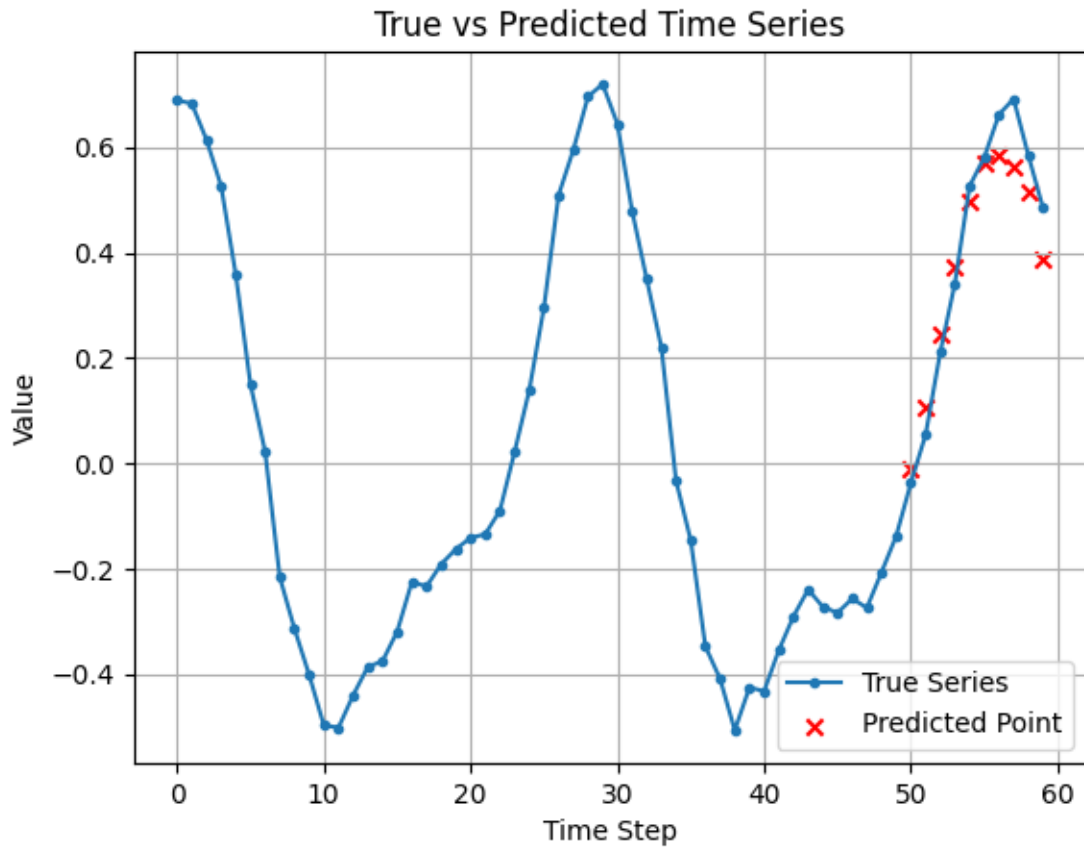
1s 14ms/step



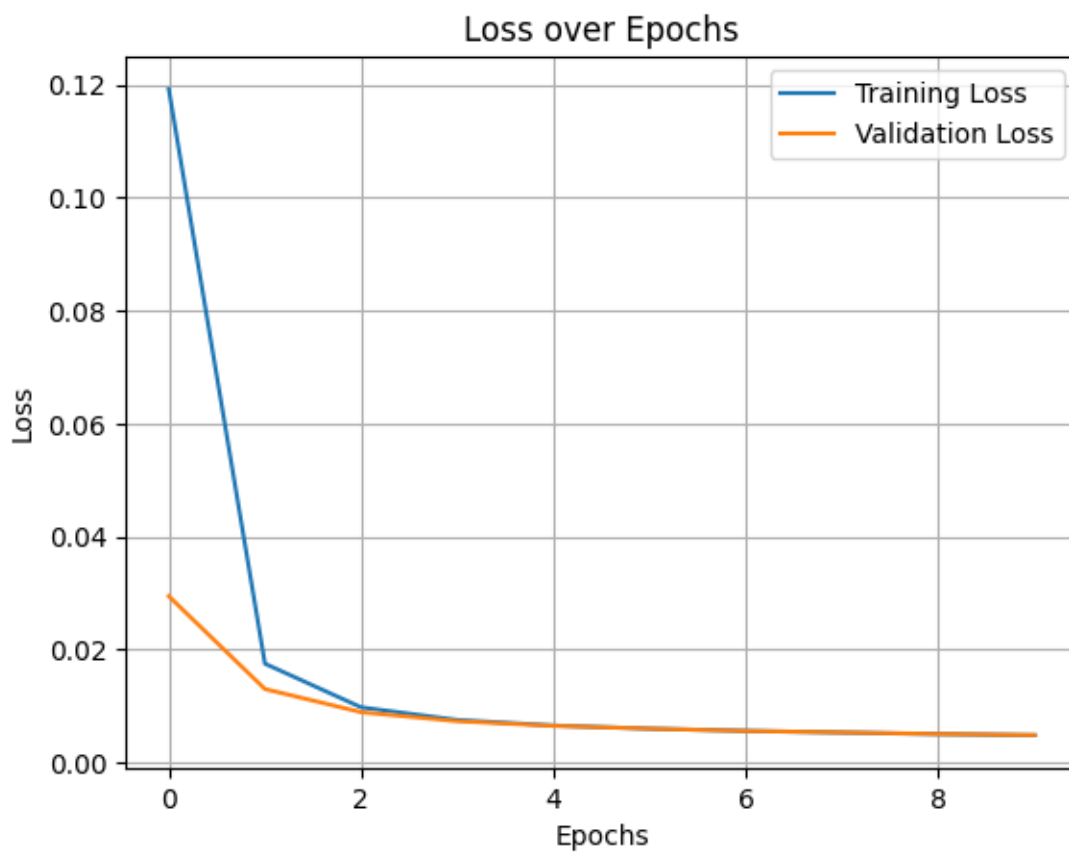
[116]: np.float32(0.0145280985)

```
[119]: plt.plot(np.hstack((X_valid[5,:,0], y_valid[5])), marker='.', label='True_
↳Series')
```

```
plt.scatter(np.arange(nstep, nstep + 10), y_pred[5, :], marker='x',
            color='red', label='Predicted Point')
plt.title('True vs Predicted Time Series')
plt.xlabel('Time Step')
plt.ylabel('Value')
plt.legend()
plt.grid()
plt.show()
```



```
[120]: plot_results(history)
```



[]: